alisation. Second, we are adding new sensors to the incubator. This includes the carbon-dioxide (CO_2) sensor mentioned above to record the release of CO_2 as the tempeh ferments. This will open up new possibilities for tracking fermentation progress, although new models will have to be developed.

12.4 The (Desktop) Robotti

The Desktop Robotti (DR) is a small physical model of the agricultural field robot Robotti developed by AgroIntelli (Fig. 12.10), for which a DT is being built using the technologies described in previous chapters. The role of the DR, and in this case the PT, is to allow a variety of tests to be carried out easily in the lab before these can be scaled up and performed in the field. The purpose of the DT is to provide services from monitoring to reconfiguration, and to enable the early testing of features intended to be part of the PT itself in the future. More specifically, the DT would be able to follow the behaviour of the robot, e.g., by predicting its location, as well as receiving its actual location from a live feed. Such functionality enables monitoring, which in turn supports the reconfiguration and adaptation of the robot in the case of anomalies. The DT would also be able to predict potential collision zones

Fig. 12.10: Real Robotti [6] (top) and the DR (bottom).

of its robot with other robots or objects in the environment and respond adequately to

prevent such collisions. The setup makes it possible to perform different experiments in a variety of scenarios, including safety-related ones, with the DR in the loop, with or without fault injection as well as model swapping. The latter consists in updating models (e.g. FMUs) at run-time without stopping and restarting the DT and PT system. The DR DT/PT system is comprised of a physical robot, a co-simulation that runs on a PC, as well as ROS2 nodes and a RabbitMQ server for communication between DT and PT (Fig. 12.11).

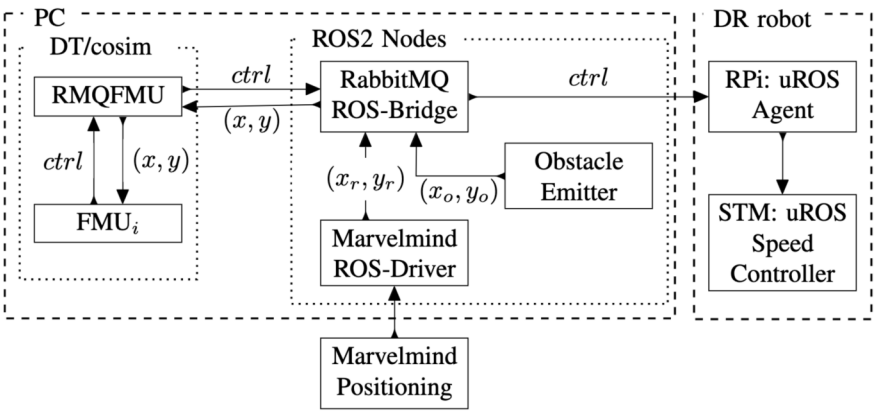

Fig. 12.11: The DR DT/PT system is distributed across the robot platform and the PC containing the co-simulation components, as well as the involved ROS2 nodes. `(x,y)` refer to position coordinates, whereas `ctrl` corresponds to control commands sent to the robot. Figure taken from [9].

12.4.1 Desktop Robotti DT Overview

As mentioned above, the DT for the DR is intended to cover functionalities from monitoring to reconfiguration, and it specifically covers the following capabilities: *parallel operation*, *fault injection with hardware in the loop*, and *runtime dynamic model swapping*. A graphical representation of the DR and its DT is given in Fig. 12.12 with the DR on the left side and the DT on the right. The following sections will refer to this figure to explain the conceptual components and characteristics of the DR and its DT. For the co-simulation of the flex-cell we use Maestro and the RabbitMQFMU [7].

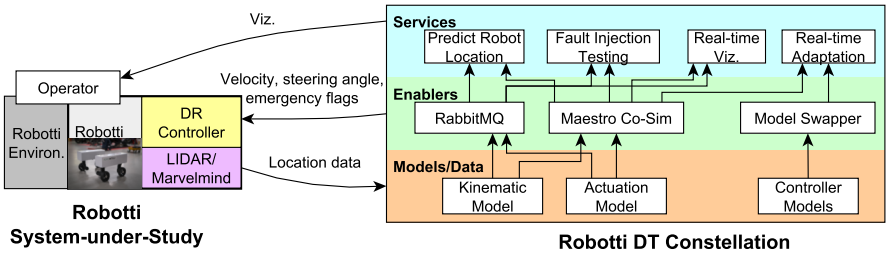

Fig. 12.12: The Desktop Robotti System-under-Study and its DT.

12.4.2 Fundamental Characteristics

This section presents the DR case study through the 14 fundamental DT characteristics as introduced in Sect. 12.2.

12.4.2.1 C1: System-under-study

The PT in this case is the DR mobile robot. The environment could be any space in which the robot is placed to drive, but is typically a structured and known laboratory. The DR has not yet been tested in an outdoor environment, where it can be subject to different weather conditions and terrain types. A human operator is also present to tele-operate and otherwise control the robot if needed. The PT is a car-like steered mobile robot, equipped with two on-board processors – a Raspberry Pi (RPi) 4B and an STM32 arm cortex-M4 based Micro-Controller Unit (MCU) (refer to the right block in Fig. 12.11). The RPi controls the steering motors and communicates with the STM32 using micro-ROS and a serial connection. Meanwhile, the MCU acts as a speed controller and can be in a driving or halted state, with the robot's movement being determined by transmitting a target speed and steering angle. The robot's position is determined by an indoor positioning system using four stationary beacons and a mobile beacon placed on the robot, which operates via a serial connection to an external modem. Lidar readings can also be used for additional information. On the PC, dockerized ROS2 nodes and a RabbitMQ server facilitate data exchange between the co-simulation and the ROS2 network. Messages relevant to the co-simulation are converted and published over RabbitMQ by a dedicated ROS2 node, with incoming messages being received and published as ROS2 messages via a subscriber node on the RPi.

12.4.2.2 C2: Acting Components

Control commands can be sent to the robot by an operator or the DT itself. These consist of velocity and steering angle used by the actuation module in the robot. The DT sends control commands if it detects an anomaly in the form of an emergency stop. The messages are sent through the RMQFMU, a RabbitMQ client encapsulated in an FMU, thereafter forwarded to the RabbitMQ server. A node on the DR side reads data from the RabbitMQ server and the forwards them to the actuation module. Note that the RMQFMU is not a custom solution for the DT of the presented robot, but rather a generic solution to bridge an FMI-based co-simulation with the external world.

12.4.2.3 C3: Sensing Components

The DR can be set up with the MarvelMind indoor positioning system, by using four beacons that serve as references for the one beacon mounted on the robot. In addition, it is possible to read data from a 2D Lidar (Rplidar A1) in the form of planar scans of the surroundings, that can be used to generate a map using the navigation stack package in ROS. As such, the robot could be tracked with respect to an initial position. In either case, the data is published to the RabbitMQ server, and transmitted to the RMQFMU residing inside the co-simulation. The RMQFMU exposes the data through its outputs, which is thereafter used to set the inputs of the interested FMUs.

12.4.2.4 C4: Multiplicities

Currently there is one DT per PT, but initial experiments have been made with two independent PTs. In this case, the DT would monitor the collision of the two (or more) robots (see Sect. 11.11) and apply a mitigation strategy by potentially triggering a mission re-plan. As an example, consider two robots that are tasked to traverse a series of rows in a field. Consider the DT predicting that these robots will collide in the i_{th} row. Then a re-plan could be triggered such that one robot continues in the i_{th} row, whereas the other robot traverses another row in the series.

12.4.2.5 C5: Data Transmitted

The robot can be operated outside of the DT by a user, through ROS topics, by providing velocity and steering commands. The DT communicates with the PT through the RabbitMQ server, and the data transmitted, which is packaged in a JSON format, could consist of velocity, steering angles and (potentially) emergency flags. In this case the RMQFMU is added to the co-simulation-based DT serving

as a bridge to the external/PT world. An example of a control command (speed and steering angles) sent to the robot by publishing to a ROS topic through the command line is given below:

```
$ ros2 topic pub -1 /robotti1/move_cmd ackermann_msgs.msg/
    AckermannDrive "{msg.speed = '20', msg.steering_angle =
    '0.8'}"
```

where a message of type *ackermann_msgs.msg/AckermannDrive* is published to the */robotti1/move_cmd* topic. On the other hand, an example of a JSON message published from the DT, i.e., from the RMQFMU, consisting in an emergency stop for the robot is given below:

```
{
 "time": "2023-06-19T13:50:07.390+00:00",
 "em-stop-flag": "True"
 }
```

where the *time* field is the timestamp associated with the message, and the *em-stop-flag* field is a Boolean flag, set to *True* to make the robot stop in case of emergency.

Should there be network disturbances (as discussed in Chap. 7) the DT/PT system will become out-of-sync. In order to deal with such circumstances, the Orchestration Synchronisation Administration (OSA) FMU [8] could be added to the co-simulation constellation. This would be able to degrade the DT to a DS by disabling the sending function of the RMQFMU, as well notifying a user of the occurrence. If new messages arrive in a bunch, it would be possible for the DT to skip to the latest messages, if the loss of messages in between can be tolerated by other models in the DT.

12.4.2.6 C6: Insights/Actions

Insights: The system can be coupled with RVIZ tool providing a visual for the location of the robot as shown in Fig. 12.13. In this case the visualisation is in 3D as described in Sect. 8.4, and physical models of the robot could also be added if they are available. The RVIZ interface offers insights into different variables, e.g., robot pose and location etc.

Actions: The DT can send control commands, or affect the behaviour of the robot, e.g., limiting its maximum speed (the third scenario described in Sect. 12.4.2.10), depending on the placed monitors.

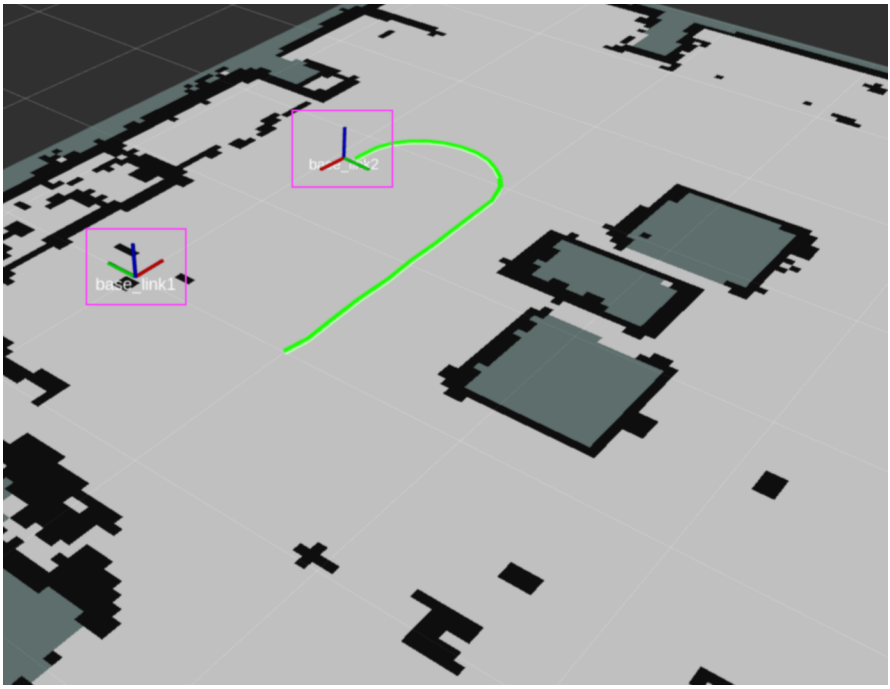

Fig. 12.13: Rviz visualisation of two DRs, one stationary, and one being tele-operated by the user (marked with the magenta rectangles). The green line depicts the path that has already been traversed by the robot.

12.4.2.7 C7: Services

The DT can be configured to support the following scenarios (independently):

1. Parallel operation – Runs kinematics/actuation models on the DT to predict robot location, thereafter used to compare with the live data. This service falls into monitoring and prediction already discussed in Chap. 9 and Chap. 10.
2. Fault injection with hardware in the loop – inject monitors or the RMQFMU data broker to perform different fault scenarios with hardware in the loop. This service falls under what-if simulations enabled with Fault Injection (FI) explored in Chap. 10.
3. Runtime model swapping – update at runtime different FMUs, like the RMQFMU, or a controller FMU. This service falls under reconfiguration capabilities during run-time covered in Chap. 10.

These scenarios are discussed further in the *Constellation* characteristic below.

12.4.2.8 C8: Enablers

The following elements are needed to support the DR DT:

1. Monitors of robot movement and distance to obstacles, that enable the DT to take action if the PT is too close to an obstacle.
2. The RMQFMU, the AMQP based rabbitMQ client encapsulated in an FMU, that enables the two way communication between DT and PT.
3. The Maestro extension for specifying FI scenarios, that allows experimentation with hardware in the loop with fault injected input/outputs.
4. The Maestro extension for model swapping, that enables swapping FMUs in the DT during run-time when new ones are made available from a user.
5. The OSA mechanism able to identify and take appropriate measures when the DT/PT are not in sync, e.g., due to network degradation, allowing operation as a DS as discussed in Sect. 1.2.2 until the network is restored.

12.4.2.9 C9: Models and Data

A kinematic and actuation model have been used to track the position of the robot. This is captured in Fig. 12.14, the bottom block, depicting how the control commands, desired speed and angle, are fed to the actuation model, which outputs the actuation acceleration, as well as angle and speed to the kinematic model. The latter computes the new position for the robot, in the form of (x,y) coordinates. The models are in the form of equations, encapsulated in python scripts, each encapsulated within a separate FMU.

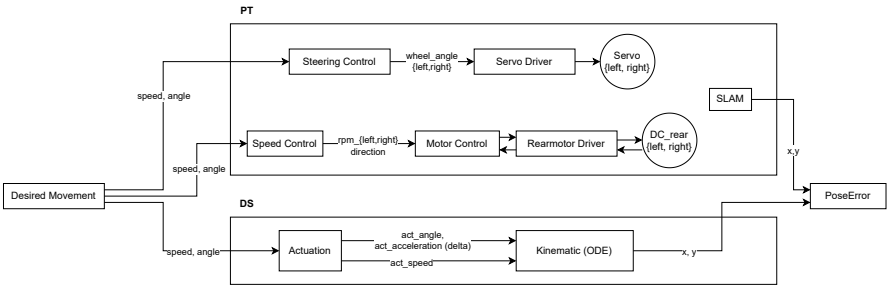

Fig. 12.14: Parallel operation scenario with the DT DR system.

The kinematics of the DR are captured by a bicycle model assuming virtual wheels in the centres of the front and rear axles, thereafter translated for the real wheels of the robot at the front and back (the reader is referred to [18] for the details of the translation). A kinematic model is physics-based (see Sect. 5.4), and uses

difference equations, computing current/future state from a past state (new position is computed from a past position). The bicycle model takes as input the velocity v and steering angle θ, and produces the change in pose as follows:

$$\dot{\psi} = v/R, \tag{12.1}$$

$$\dot{x} = v \cdot cos\psi, \tag{12.2}$$

$$\dot{y} = v \cdot sin\psi, \tag{12.3}$$

where $\dot{\psi}$, $\dot{x}$, $\dot{y}$ represent the change in heading, x, y positions respectively.

The actuation model captures the behaviour of the DC motors, modelled as a first order system. Since DC motors are controlled by some input voltage, resulting in a mechanical output, there is no direct correlation to speed. As such a calibration procedure is carried out to compute the calibration coefficients for the model (see Chap. 6). To this end, a Proportional-Integral regulator was implemented on a microcontroller to manage the speed of DC motors within the Desktop Robotti. The selected regulator imparts a type 1 system characteristic, ensuring a steady-state error in response to step inputs. Subsequently, the model, predicated upon a sampling rate of 60 Hz, undergoes discrete-time conversion and is seamlessly incorporated into the co-simulation platform, with attention given to considerations of stability and phase margins.

12.4.2.10 C10: Constellation

In the following, the DT/PT setup is shown for each scenario.

Scenario 1: Parallel Operation

The components of the system in this experiment are shown in overview in Fig. 12.15. The most significant elements here are the models used in the co-simulation environment to estimate where the robot is, given initial conditions, velocity and steering angle. The results of short experiments performed with the DR are shown in Fig. 12.16, where it is possible to visually compare the actual and predicted positions of the robot.

Scenario 2: Fault Injection

Two fault injection experiments are performed to simulate issues in the components (e.g., monitors) as well as corruption in the data, which would lead to inaccurate

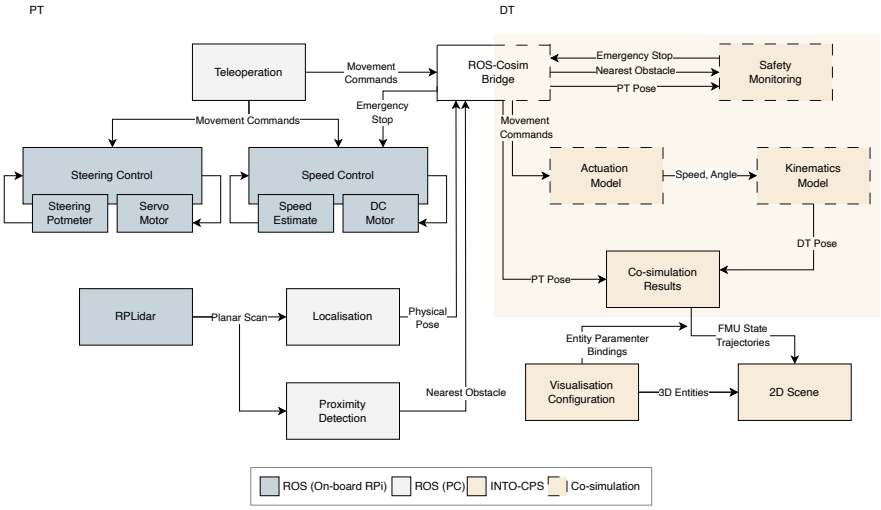

Fig. 12.15: DT/PT system for the parallel operation scenario. The shaded area represents the DT. Figure taken from [26].

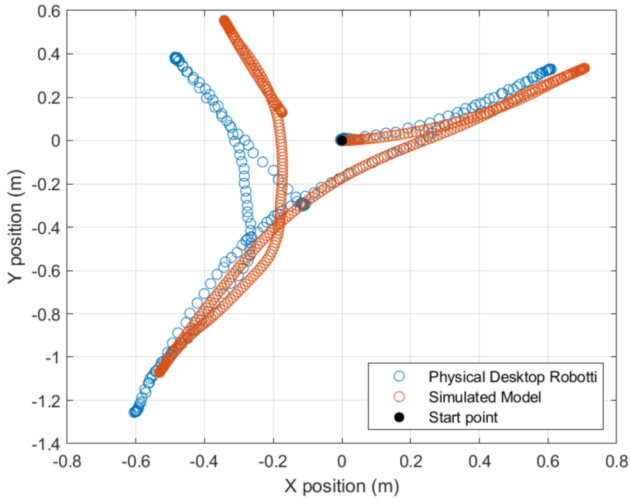

Fig. 12.16: Actual and predicted positions of the DR during experiment. Figure taken from [26].

evaluations in the monitors. The monitors used here check the distance to a virtual obstacle placed in the path of the DR, and set an emergency stop flag if the distance drops below a configurable threshold. Note that there is a redundancy of monitors

to support fault-tolerance, one at the PT and the other at the DT level, referred to as PTM and DTM respectively. Should the PTM fail, the DTM might still be able to send an emergency stop if the PT gets too close the obstacle. To perform tests under faulty conditions, the PTM is encapsulated as an FMU and placed within the DT, such that the fault injection techniques described in Sect. 10.2 can be applied.

In the first experiment, a fault is injected into the data broker, i.e., RMQFMU, signifying bad data coming from the PT (see Fig. 12.17). As the data is corrupt, both monitors (DT and PT ones) will not output an emergency stop when needed. However, it is possible to imagine a situation in which correct information comes to the DT from other sources, e.g., an operator or other DTs, and can be thus used from the DT to produce the desired output (the emergency stop). This source of correct information is captured by the Correct Information (CI) component.

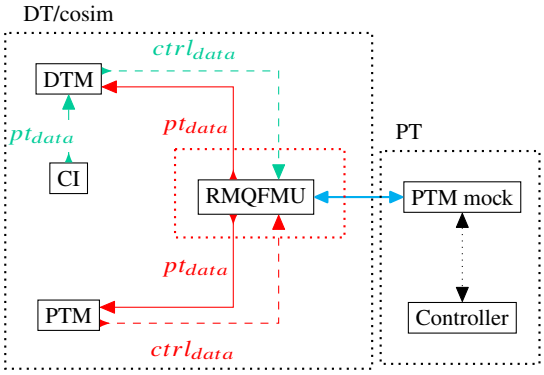

Fig. 12.17: Fault injecting the data broker (RMQFMU) to test the fault tolerance of the DT through the DT Monitor (DTM) when a source of correct information is available (CI). Figure taken from [9].

In the second experiment, it is desirable to test the fault tolerance capabilities of the DTM, should the PTM fail due to some internal crash. Similarly to the previous experiment, to perform fault injection on the PTM within the DT ecosystem, the monitor first needs to be encapsulated as an FMU. Thereafter, the PTM can be included in the DT, and its output can be tampered with by the fault injection mechanism. The faulty behaviour consists of an emergency stop not being issued when the distance to the obstacle is less than a predefined threshold. The system is depicted in Fig. 12.18, where the DTM is the DT monitor, and the PTM is the PT monitor temporarily placed in the DT for testing.

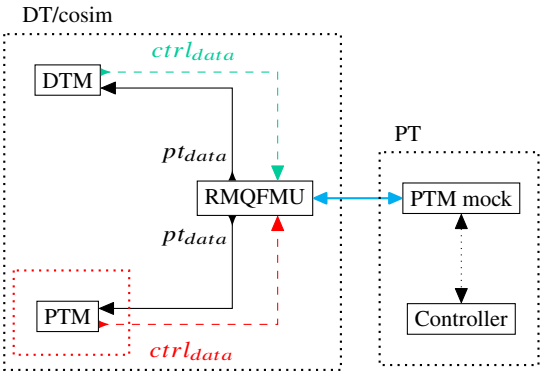

Fig. 12.18: Fault injecting the PT Monitor (PTM) to test the fault tolerance of the DT through the DTM. Figure taken from [9].

Scenario 3: Model Swapping

A *model swapping* technique enables the DT to reconfigure itself when new models become available, without the need to shut down the system and restart it as a whole. The basic DT setup consists of the data broker (RMQFMU), an actuation, and a kinematic model. Two examples of run-time swapping are considered below.

In our first example, a new FMU, specifically the OSA FMU that deals with the detection of time discrepancies between DT and PT, should be inserted. Note that this action would change the existing structure of FMUs. Furthermore, in order to support

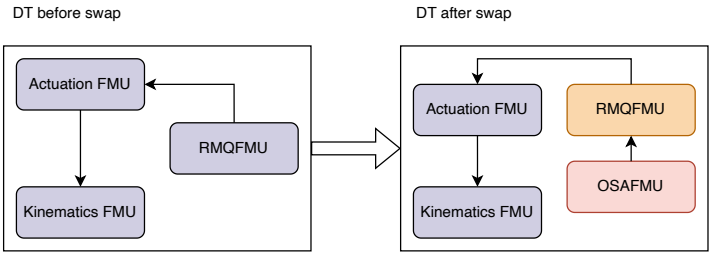

Fig. 12.19: Model swapping by replacing the RMQFMU with a newer version and introducing a new FMU (OSA) with time discrepancy detection capabilities.

the new functionality provided by the OSA FMU, the RMQFMU should be updated as well to include the feature of disabling sending of data if a time discrepancy is discovered. In other words, allowing the transition from DT to DS as discussed in Sect. 1.2.2 for as long that the discrepancies are present. The constellation of the

system is depicted in Fig. 12.19. The results of this swap under different conditions are visualised in Fig. 12.20, specifically the values of the steering angle that is fed to the RMQFMUs. The different conditions refer to different stepping and swapping conditions placed for the RMQFMUs. Specifically, when the step condition is satisfied, the inputs of the FMU to be swapped become active. This is useful when these inputs should be synchronised to the outputs of other FMUs. When the swap condition is satisfied, the outputs of the FMU to be swapped are activated, thus the swapping is complete, with the old FMU being discarded. From the tests it can seen that in order to swap in a safe way, i.e., without discrepancies in the steering angles, the new RMQFMU should be stepped only when its initial output matches that of the old RMQFMU. Note also that the new RMQFMU will not have access to the messages already acknowledged by the RabbitMQ server, which are being internally processed by the old one clearly visible in Fig. 12.20a.

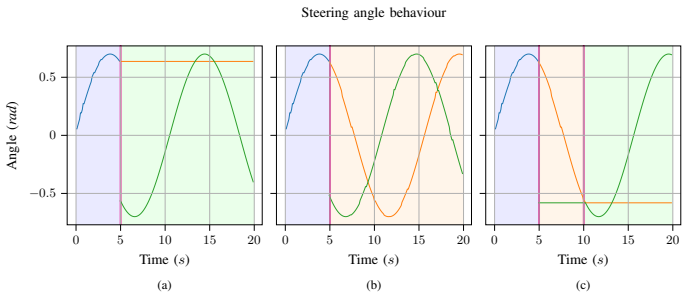

Fig. 12.20: (a) In the first run the FMUs are swapped instantly at time 5s. At that time the old RMQFMU (blue and orange lines) is not stepped any longer (orange line), whereas the new one (green line) starts being stepped. (b) In the second run, a condition is placed for swapping the FMUs, that corresponds to when the steering angles produced by the FMUs match. Note that this never happens, the sinusoids are displaced in phase. (c) In the third run, a stepping condition is placed on the new FMU, so that it cannot step until the outputs (steering angles) between FMUs match. The swap condition is the same as the stepping condition, therefore the swap happens when the outputs have matched. Figure taken from [2]

In our second example, an existing controller is updated with a new one that applies a speed cap to the speed provided by a user (Fig. 12.21). As can be seen in Fig. 12.22, the new controller is already available at time 6s, however the swap only takes place after the robot has reduced its speed under a predefined safety threshold. After that time, even if the user inputs a higher speed than the cap, the new controller will not allow said speed to be propagated to the robot. It will send the cap itself.

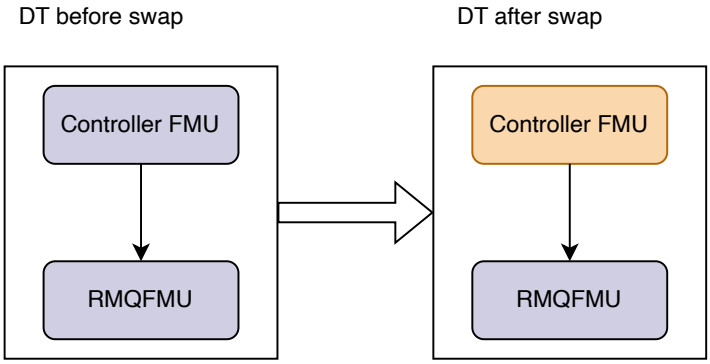

Fig. 12.21: Swapping controllers for the DR.

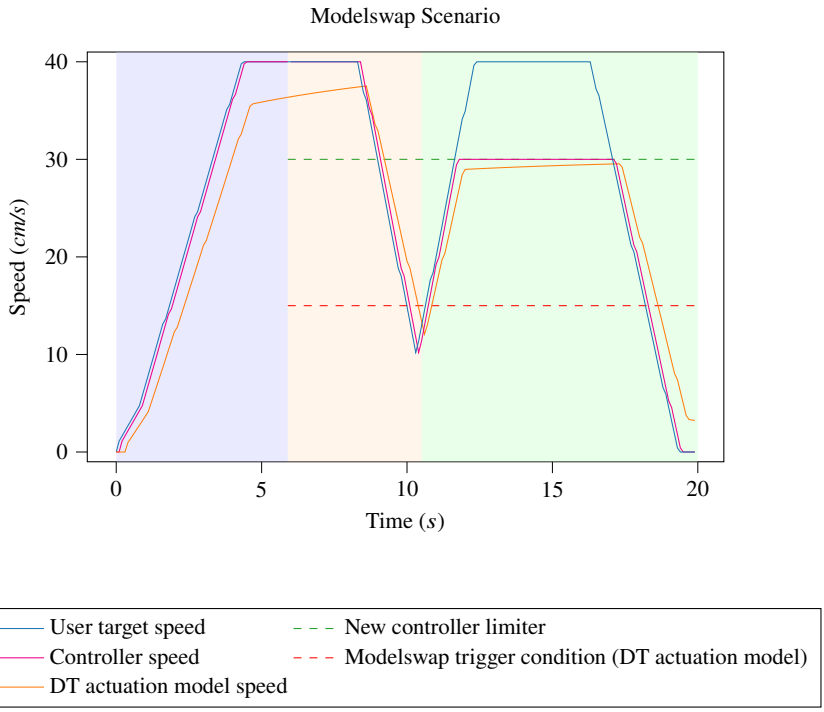

Fig. 12.22: Model swapping by replacing an existing controller with a new one that allows placing a cap on the speed of the DR. Figure taken from [2].

12.4.2.11 C11: Time-Scale

The model inside the DT can run faster than real-time, and while it is expected that it does so, there are currently no guarantees that it will. The speed of execution can be halted or slowed down by the rate of data coming in the DT through the RMQFMU. This is due to the RMQFMU requiring a constant feed of data in order to step. Any delay in this feed will leave the RMQFMU in a waiting mode until a certain timeout is reached, thus resulting in the whole DT being stuck.

In more detail, the Robotti DT operates at these time-scales:

- **Slower-than-real-time:** The services that need to go over the DT Manager and generate an output based on PT and DT data.
- **Real-time (best effort):** control commands, data recording, and visualisation.
- **Faster-than-real-time (best effort):** co-simulation based DT.

12.4.2.12 C12: Fidelity Considerations

This case requires medium to high-fidelity models, as they would be used to predict anomalies and divergences between predicted and actual robot positions, and possibly collision zones. If the models are not of sufficient fidelity, the DT cannot be trusted to send commands to the robots, as it might try to apply mitigation strategies where they are not needed, e.g., by stopping the robots as a precaution. On the other hand, it might react too late or not at all to dangerous situations, e.g., collisions, that would cause damage to the robots themselves, and perhaps others situated nearby.

With respect to this, a known issue is present in the kinematic model described earlier. There, the change in angle is assumed to happen instantaneously, which has contributed to the divergences observed in Fig. 12.16.

12.4.2.13 C13: Life-cycle Stages

The DT can be used throughout the *development* (consider different levels of FI simulations, depending on what hardware is available), and through *operation* (consider that FMUs used only for testing can be removed in the deployed DT, leaving only what is needed for the operation).

12.4.2.14 C14: Evolution

The evolution of the DR DT has been reported in several publications. The initial attempts, focused on the setup of the parallel operation [26]. Thereafter, the enhancement of the DT with fault injection capabilities was explored with the DR in the loop [10, 9]. The DT/PT system is affected by the quality of the communication, or

lack thereof, which has an impact on the execution of the DT. To tackle this issue, the DT was extended with time discrepancy detection capabilities, allowing it to degrade to DS when there was a degradation in communication as discussed in Sect. 1.2.2 [8]. Finally, the DT was extended further with a model swapping capability, enabling it to swap models at run-time under configurable conditions [2].

12.4.3 Summary and Future Work

The DR case-study has shown proof-of-concept DTs for a mobile robot, covering functionalities such as: (i) parallel operation, where the DT continuously predicts where the robot should be as well as having access to the actual position data, thus laying the groundwork for monitoring and fault detection; (ii) fault injection, where different components in the DT can be injected with faulty values allowing different kinds of experiments with hardware in the loop, as close to the operational stage as possible; (iii) model swapping, where the DT is able during run-time to swap components, given that certain conditions – specified by the user – are satisfied.

The main challenge is related to the simulation-reality gap, and the development of high-fidelity models. Future work will focus on this aspect, as well as enhancing the DT to deal with a multi-robot system, providing functionalities like the evaluation and mitigation of collision zones between robots during deployment. Additionally, fault detection and diagnosis shall be explored, with relevant techniques to be implemented in the DT such that scenarios where, e.g., a robot gets a flat tire, can be identified and handled properly. Finally, the fault injection framework could be extended to allow for fault injection at the DT and the PT levels simultaneously, allowing for richer experimental setups for testing the adopted fault tolerance mechanisms. Fault injection could also be used in the context of what-if simulations, by providing possibly useful insights to the fault diagnostics.

12.5 The Flex-cell

The flex-cell is a manufacturing cell composed of two robotic arms used for cooperative assembly. A key feature is that the two arms are different and from diverse manufacturers. As an academic DT engineering case study, it provides opportunities for researchers to explore:

- composable DTs;
- cooperative systems with synchronous motions and coupled models;
- multiple abstraction levels of attributes and operations of both robots and grippers;